Guía de pruebas del swarm PTaaS

UTN FRVM · Seguridad Informática 2026 · Grupo 03 · Práctico final integrador

Esta guía prueba el swarm de punta a punta, en cuatro niveles que van de menos a más infraestructura. Cada paso dice **qué hacer**, el **comando** y **qué tenés que ver**. No pases al paso siguiente hasta que el actual dé lo esperado.

Los resultados esperados son los que se obtuvieron al probarlo el 2026-11-03, en Windows 11 con Docker Desktop 28.5.1.

Nivel	Qué prueba	Necesita	Tiempo
1	La lógica del swarm	Python	1 segundo
2	Que el sandbox esté aislado	Docker Desktop	2 minutos
3	La corrida real contra phantomcorp	Docker Desktop + lab	5 minutos
4	Las demostraciones para el informe	Lo del nivel 3	3 minutos

Los niveles 3 y 4 también se pueden hacer **sin consola**, desde el panel web del swarm (Parte F). La consola sigue haciendo falta para levantar el lab y el panel.

Parte A — Preparación

Paso 1 — Abrir Git Bash

Todos los comandos de esta guía son para **Git Bash**, no para PowerShell ni para `cmd`. En el menú Inicio, buscá "Git Bash" y abrilo.

Los comandos largos están partidos en varias líneas que terminan en `\`. Copialos **completos, con todas sus líneas juntas**, y pegalos de una vez: Git Bash los toma como un solo comando.

Paso 2 — Ir a la raíz del repo

Reemplazá la ruta por la carpeta donde clonaste el repo:

```
cd /c/Users/USUARIO/Documents/proyectos/LabsSeguridadInformatica2026
```

Comprobá que estás en el lugar correcto:

```
ls entregas/labfinal/grupo03/ptaas
```

Tenés que ver: `Dockerfile.runner`, `PRUEBAS.md`, `PRUEBAS.pdf`, `agentes.py`, `bitacora.py`, `orquestador.py`, `panel.html`, `panel.py`, `roe.py`, `sandbox.py` y `test_ptaas.py`.

Si dice "No such file or directory": no estás en la raíz del repo, o tu copia no tiene la rama con la entrega.

Paso 3 — Verificar Python

```
python3 --version
```

Tenés que ver: `Python 3.9` o superior. El código usa sólo la biblioteca estándar: no hay nada que instalar con `pip`.

Paso 4 — Verificar openssl

```
openssl version
```

Tenés que ver: una versión de OpenSSL. Git Bash ya trae una. Si no aparece, el nivel 1 igual funciona, pero se saltea la verificación independiente.

Parte B — Nivel 1: la lógica del swarm (sin Docker)

Este nivel no usa Docker ni red: el sandbox se reemplaza por uno falso que devuelve salidas fijas. **Cada integrante del grupo lo corre en su PC antes de commitear.**

Paso 5 — Correr el test

```
python3 entregas/labfinal/grupo03/ptaas/test_ptaas.py
```

Tenés que ver:

```
cadena verificada también con openssl
OK
```

Qué comprobó:

- **Determinismo:** dos corridas iguales dan la bitácora idéntica, byte a byte.
- **RoE:** un agente que intenta salirse del alcance, inyectar comandos, usar una herramienta no permitida o un puerto inválido nunca llega al sandbox, y cada intento queda registrado como `rechazada`.
- **Integridad:** alterar una línea de la bitácora, truncarla o usar otra clave se detecta.
- **Fuente independiente:** `openssl` recalcula la misma cadena HMAC, así que el código hace de verdad HMAC-SHA256 y no sólo coincide consigo mismo.

Si falla: el mensaje del `AssertionError` dice qué propiedad se rompió.

Parte C — Nivel 2: el sandbox está aislado (Docker, sin target)

Paso 6 — Abrir Docker Desktop

Abrí Docker Desktop y esperá a que diga **"Engine running"** abajo a la izquierda. Después comprobalo desde Git Bash:

```
docker info --format '{{.ServerVersion}}'
```

Tenés que ver: un número de versión, por ejemplo `28.5.1`.

Si dice "error during connect": Docker Desktop todavía no terminó de arrancar. Esperá y repetí.

Paso 7 — Construir la imagen del sandbox

```
docker build -t ptaas-runner \
  -f entregas/labfinal/grupo03/ptaas/Dockerfile.runner \
  entregas/labfinal/grupo03/ptaas
```

Tenés que ver: que termina sin errores. La primera vez descarga Alpine y tarda alrededor de un minuto; las siguientes, segundos.

Paso 8 — Desactivar la conversión de rutas de Git Bash

Git Bash convierte las rutas como `/tmp` o `/proc/...` en rutas de Windows antes de pasárselas a `docker`, y los comandos de los pasos siguientes fallan con errores raros (por ejemplo `mount path must be absolute`). Esto lo evita:

```
export MSYS_NO_PATHCONV=1
```

No imprime nada. Vale hasta que cierres la terminal: si la cerrás, repetí este paso. `sandbox.py` no tiene este problema, porque Python llama a `docker` sin pasar por Git Bash.

Paso 9 — Comprobar que no corre como root

```
docker run --rm --cap-drop ALL --read-only --user 1000:1000 ptaas-runner id
```

Tenés que ver: `uid=1000(runner) gid=1000(runner) groups=1000(runner)`

Paso 10 — Comprobar que el sistema de archivos es de solo lectura

```
docker run --rm --read-only --user 1000:1000 ptaas-runner touch /x
```

Tenés que ver: `touch: /x: Read-only file system`. Que falle es lo correcto.

Paso 11 — Comprobar que no tiene capabilities

```
docker run --rm --cap-drop ALL ptaas-runner grep CapEff /proc/self/status
```

Tenés que ver: `CapEff: 0000000000000000` (todo en cero: ninguna capability).

Paso 12 — Comprobar que no puede escalar privilegios

```
docker run --rm --security-opt no-new-privileges ptaas-runner \
  grep NoNewPrivs /proc/self/status
```

Tenés que ver: `NoNewPrivs: 1`

Paso 13 — Comprobar que nmap funciona con todas las restricciones

Es la prueba más importante del nivel: usa exactamente las mismas restricciones que `sandbox.py`.

```
docker run --rm --cap-drop ALL --security-opt no-new-privileges \
  --read-only --tmpfs /tmp --user 1000:1000 \
  --pids-limit 64 --memory 256m --cpus 0.5 \
  ptaas-runner nmap -sT -sV -Pn -p 1 127.0.0.1
```

Tenés que ver: que termina con una línea `Nmap done: 1 IP address (1 host up)`.

Si falla con `could not locate nse_main.lua`**:** a la imagen le falta el paquete `nmap-scripts`, que `-sV` necesita. Revisá `Dockerfile.runner` y volvé al paso 7.

Parte D — Nivel 3: corrida real contra phantomcorp

Paso 14 — Levantar el lab final

En Windows no hay `make`, así que `./ctf lab final` y `make lab N=final` fallan. Este es el mismo comando que corre el Makefile:

```
docker compose -p cyberlab --project-directory . \
  -f entorno/docker-compose.yml \
  -f labs/labfinal-practico-integrador/entorno/target.compose.yml \
  up -d --build
```

Tenés que ver: al final, `Container cyberlab-phantomcorp Started` y `Container cyberlab-attacker Started`.

Si antes levantaste otro lab, puede aparecer otro contenedor como `Recreate` (por ejemplo `cyberlab-ssrf-web`) o un aviso `Found orphan containers`: es normal, son restos del otro lab y no afectan al lab final.

Paso 15 — Comprobar que los contenedores están corriendo

```
docker ps --format '{{.Names}}  {{.Status}}'
```

Tenés que ver: `cyberlab-phantomcorp` y `cyberlab-attacker`, los dos con estado `Up`.

Paso 16 — Comprobar que existe la red del lab

```
docker network ls --filter name=cyberlab
```

Tenés que ver: una fila con `cyberlab_labnet`. Es la red a la que `sandbox.py` conecta cada contenedor.

Paso 17 — Crear la carpeta de evidencia

```
mkdir -p .loot
```

`.loot/` está en el `.gitignore`: lo que se guarda ahí no se sube al repo. Es la carpeta que el enunciado pide para la evidencia.

Paso 18 — Generar la clave HMAC de la bitácora

```
export PTAAS_CLAVE="$(python3 -c 'import secrets; print(secrets.token_hex(32))')"
```

No imprime nada. Para ver la clave y guardarla:

```
echo "$PTAAS_CLAVE"
```

Guardala fuera del repo (por ejemplo, en un gestor de contraseñas):

- Sin la clave, la bitácora ya no se puede verificar.
- La variable se pierde si cerrás la terminal. Para volver a cargarla: `export PTAAS_CLAVE='la-clave-que-guardaste'`
- Nunca la guardes en un archivo del repo: el CI de la cátedra rechaza los secretos.
- En PowerShell, la variable se define con `$env:PTAAS_CLAVE = '...'`.

Paso 19 — Correr el swarm

```
python3 entregas/labfinal/grupo03/ptaas/orquestador.py .loot/bitacora-1.jsonl
```

Tarda unos **30 segundos**. **Tenés que ver** tres líneas:

- `puertos:` los puertos abiertos del target, por ejemplo `{80: 'http?'}`. El `?` significa que nmap no está seguro del servicio.
- `rutas:` las rutas que respondieron algo distinto de 404. Con la versión actual tenés que ver `(80, '/portal-rrhh'): 200`: es el portal del hito R1.

Enum lee primero `/robots.txt` y prueba también las rutas que ese archivo esconde (`Disallow`). Por eso `/portal-rrhh` aparece aunque no esté en ninguna lista fija: el sitio la menciona en `robots.txt`. - `cabeza de la bitácora:` un hash de 64 caracteres hexadecimales.

Si dice `FileExistsError`: ese archivo de bitácora ya existe. El orquestador nunca pisa una bitácora anterior: usá otro nombre, por ejemplo `.loot/bitacora-4.jsonl`.

Si dice `definí PTAAS_CLAVE`: la variable se perdió. Volvé al paso 18.

Paso 20 — Anotar la cabeza de la bitácora

Copiá el hash de `cabeza de la bitácora:` al informe. Es la evidencia de que la bitácora no se modificó después de la corrida.

Paso 21 — Verificar la bitácora

```
python3 entregas/labfinal/grupo03/ptaas/bitacora.py .loot/bitacora-1.jsonl
```

Tenés que ver: exactamente el mismo hash del paso 20.

Paso 22 — Si la corrida vino vacía: revisar la bitácora

Hacé este paso sólo si en el paso 19 `puertos:` salió `{}`.

```
python3 -c "
import json
for linea in open('.loot/bitacora-1.jsonl', encoding='utf-8'):
    e = json.loads(linea)
    print(e['paso'], e['evento'], str(e['datos'])[:200])
"
```

Cada acción tiene un evento `propuesta` (el comando) y uno `resultado` (con `rc` y `salida`):

- `rc: -1` con `salida: timeout`: la herramienta superó los 60 segundos del sandbox. Contra phantomcorp, `nmap -sV` completo tarda unos 90 segundos; por eso `roe.py` usa `--version-light`, que tarda unos 11.
- `rc` distinto de 0 con un error de Docker: revisá los pasos 14 a 16.

Parte E — Nivel 4: demostraciones para el informe

Paso 23 — Hacer una copia de la bitácora

Las alteraciones se hacen siempre sobre una **copia**, para no romper la evidencia original:

```
cp .loot/bitacora-1.jsonl .loot/alterada.jsonl
```

Paso 24 — Alterar la copia

Este comando cambia un resultado `404` por `200` en la línea 6 de la copia, como si alguien quisiera inventar un hallazgo:

```
sed -i '6s/"salida": "404"/"salida": "200"/' .loot/alterada.jsonl
```

Comprobá que la copia cambió:

```
diff .loot/bitacora-1.jsonl .loot/alterada.jsonl
```

Tenés que ver: `6c6` y dos versiones de la línea 6, una con `"404"` y otra con `"200"`. Si `diff` marca todas las líneas, la bitácora se generó con una versión anterior del código que escribía fines de línea de Windows: generá una nueva con el paso 19. Si `diff` no muestra nada, la línea 6 no tenía un 404: cambiá el `6` del paso anterior por otra línea par (8, 10, 12...).

Paso 25 — Verificar la copia alterada

```
python3 entregas/labfinal/grupo03/ptaas/bitacora.py .loot/alterada.jsonl
```

Tenés que ver: `ValueError: cadena rota entre las líneas 6 y 7`. Que falle es lo correcto: la alteración se detectó.

Paso 26 — Verificar con una clave equivocada

```
PTAAS_CLAVE=otra-clave python3 \
  entregas/labfinal/grupo03/ptaas/bitacora.py .loot/bitacora-1.jsonl
```

Tenés que ver: `ValueError: cadena rota entre las líneas 1 y 2`. Sin la clave correcta, nadie puede rehacer la cadena.

Paso 27 — Hacer una segunda corrida

```
python3 entregas/labfinal/grupo03/ptaas/orquestador.py .loot/bitacora-2.jsonl
```

Tarda unos 30 segundos, igual que en el paso 19.

Paso 28 — Comparar las decisiones de las dos corridas

```
diff <(grep -o '"argv": \[[^]]*\]' .loot/bitacora-1.jsonl) \
     <(grep -o '"argv": \[[^]]*\]' .loot/bitacora-2.jsonl) \
  && echo "mismas decisiones"
```

Tenés que ver: `mismas decisiones`.

La **cabeza** de las dos bitácoras va a ser distinta, y es correcto: la salida de nmap incluye la fecha y la duración del escaneo. Contra el target real, lo determinista son las **decisiones** del swarm; la igualdad byte a byte la garantiza el nivel 1, donde las salidas son fijas.

Paso 29 — Borrar la copia alterada

```
rm .loot/alterada.jsonl
```

Parte F — El panel web (lo mismo, sin consola)

El panel hace desde el navegador lo mismo que los niveles 3 y 4, y además lo muestra: correr el swarm y verlo en vivo en un diagrama, reproducir una corrida paso a paso, poner a prueba la RoE, verificar la bitácora, demostrar una alteración y correr el test. Usa el mismo código que la consola; no hay una segunda implementación.

Paso 30 — Levantar el panel

Antes, Docker Desktop tiene que estar abierto (paso 6) y el lab levantado (paso 14). Después:

```
python3 entregas/labfinal/grupo03/ptaas/panel.py
```

Tenés que ver: `Panel del swarm en http://127.0.0.1:8765/` en la terminal, y el navegador se abre solo en esa dirección. Si no se abre, abrila a mano.

La terminal queda ocupada mientras el panel está abierto: no la cierres. Para los demás comandos, abrí otra ventana de Git Bash.

Si dice `address already in use` **o** `10048`**:** ya hay un panel abierto en otra terminal. Cerralo con Ctrl+C y repetí este paso: un panel abierto antes de actualizar el código sigue corriendo la versión vieja.

Paso 31 — Cargar la clave HMAC

En la tarjeta **1 · Clave HMAC**, tocá **Generar** (o pegá la clave del paso 18). Tocá **Copiar** y guardala fuera del repo: el panel no la guarda, y si recargás la página hay que volver a cargarla.

Paso 32 — Correr el swarm desde el panel

En la tarjeta **2 · Correr el swarm**, el nombre de la bitácora ya viene completado con la fecha y la hora. Tocá **▶ Correr el swarm**.

Tenés que ver, en la columna de la derecha, mientras corre:

- **Cómo viaja cada acción:** un diagrama con cuatro cajas (Agentes → Orquestador → Sandbox → phantomcorp). Se ilumina el recorrido de la acción que está en curso: hacia la derecha cuando un agente propone algo, hacia la izquierda cuando vuelve la respuesta. Abajo, un texto explica en castellano qué está pasando.
- **Lo que hizo el swarm:** una tarjeta por acción, con el agente, qué hizo ("Probó si existe la página /admin") y qué pasó ("No existe (HTTP 404)"). Las acciones que encontraron algo se marcan en verde.
- **Pizarra:** lo que los agentes ya saben del target (puertos abiertos y páginas probadas).

A los 30 segundos, más o menos, aparece un recuadro verde **Corrida terminada** en la tarjeta 2, con los puertos, las páginas encontradas y la cabeza de la bitácora. Anotá la cabeza para el informe.

Si aparece un recuadro rojo: el mensaje dice qué falta. Los dos casos habituales son que Docker no esté corriendo (volvé al paso 6) o que no exista la red del lab (volvé al paso 14).

Para ver el dato crudo: cada tarjeta tiene "Ver lo técnico". Ahí están los dos *eventos* que esa acción dejó en la bitácora: la **propuesta** (el comando exacto que pidió el agente) y el **resultado** (la respuesta tal cual llegó, con su `rc`, el código de salida: 0 quiere decir que el comando terminó bien, no que encontró algo).

Paso 33 — Reproducir una corrida paso a paso

Debajo del diagrama, tocá **▶ Reproducir**. El panel vuelve a mostrar la bitácora elegida desde el principio, un evento cada dos segundos, con el diagrama y el texto explicativo. Con **◀** y **▶|** se avanza a mano, y con **Ver todo** se vuelve a la vista completa.

No necesita Docker: sirve para presentar una corrida que ya se hizo.

Paso 34 — Poner a prueba la RoE desde el panel

En la tarjeta **3 · Poner a prueba la RoE**, tocá **Lanzar el agente pícaro**. Es un agente real que propone cuatro acciones prohibidas: atacar `8.8.8.8`, colar un comando en una ruta, usar `bash` y usar un puerto inválido. No necesita Docker.

Tenés que ver: `✓ La RoE bloqueó todo`, el contador **bloqueadas por la RoE** en 4 y cuatro tarjetas rojas, cada una con el motivo del bloqueo. Si tocás **▶ Reproducir**, el diagrama muestra el orquestador en rojo y la acción sin llegar nunca al sandbox ni al target.

Paso 35 — Verificar la bitácora desde el panel

En la tarjeta **4 · Verificar una bitácora**, elegí la bitácora del paso 32 y tocá **Verificar cadena**.

Tenés que ver: `✓ Nadie modificó esta bitácora` y la misma cabeza del paso 32. Si elegís una bitácora de otra corrida, hecha con otra clave, tiene que decir `✗ La cadena no verifica`: es lo correcto.

Paso 36 — Demo de alteración desde el panel

Con la misma bitácora elegida, tocá **Demo de alteración**.

Tenés que ver: `✓ La alteración se detectó`, la línea que se cambió (por ejemplo `404 → 200`) y en qué líneas se rompe la cadena. La demo trabaja sobre una copia temporal: la bitácora original no se modifica.

Paso 37 — Correr el test desde el panel

En la tarjeta **Test del swarm**, tocá **Correr el test**.

Tenés que ver: `✓ El test pasó`, con la misma salida del paso 5.

Paso 38 — Cerrar el panel

En la terminal donde corre el panel, apretá **Ctrl+C**.

Cómo se protege el panel

- Escucha sólo en `127.0.0.1`: no es accesible desde otra computadora.
- Cada pedido lleva un token que sólo conoce la página del panel, así que otra página abierta en el navegador no puede ordenarle una corrida (CSRF).
- Rechaza pedidos con un `Host` ajeno, lo que frena el DNS rebinding.
- El target no se elige en el panel: es el de `roe.py`, y toda acción sigue pasando por la RoE.
- `test_ptaas.py` comprueba estos controles en cada corrida.

Parte G — Cierre

Paso 39 — Guardar la evidencia para el informe

Para el informe hacen falta:

- La salida del paso 5 (test del nivel 1).
- La salida de los pasos 9 a 13 (aislamiento del sandbox).
- La salida del paso 19 y la cabeza anotada en el paso 20.
- La salida de los pasos 25, 26 y 28 (integridad y reproducibilidad).
- Si usaron el panel, capturas de los pasos 32, 34, 35 y 36.
- Las bitácoras de `.loot/`, que no se suben al repo pero respaldan lo anterior.

Paso 40 — Bajar el lab

```
docker compose -p cyberlab --project-directory . \
  -f entorno/docker-compose.yml \
  -f labs/labfinal-practico-integrador/entorno/target.compose.yml \
  down
```

Tenés que ver: `Container cyberlab-phantomcorp Removed` y `Container cyberlab-attacker Removed`.

Alcance actual

Los agentes que existen llegan hasta **R1**: encuentran los puertos abiertos, las páginas de una lista fija y las que `robots.txt` esconde. El swarm resuelve R1 solo, sin pasos manuales.

Cada agente nuevo para R2–R4 necesita primero su caso en `test_ptaas.py`, con salidas fijas del sandbox falso, y recién después se prueba contra el target con esta guía.